

NumPy — Complete Study Guide

This guide covers everything in your NumPy notebook, cleaned up, corrected, completed (including the parts left unfinished), and explained clearly.

1. What is NumPy and Why Does It Matter for Data Science/ML?

NumPy (“Numerical Python”) is the foundation library for numerical computing in Python. Almost every other Data Science library — including **Pandas**, **Matplotlib**, and **Scikit-learn** — is built on top of it.

Why it matters: - **Converts data into numbers** — Machine Learning models only understand numbers, so any data (text, images, categories) eventually gets converted into numeric arrays for preprocessing. - **Broadcasting** — NumPy can apply operations across entire arrays at once without writing manual loops (much faster than plain Python). - **Arrays** — The core data structure in NumPy is the **array** — an efficient, organized arrangement of data (unlike Python lists, arrays are fast and support vectorized math).

```
import numpy as np
```

2. Creating Arrays and Their Core Properties

NumPy arrays can have any number of **dimensions**:

```
a1d = np.array([1, 2, 3]) # 1D
array (a vector)
a2d = np.array([[1, 2, 3], [4, 5, 6]]) # 2D
array (a matrix)
a3d = np.array([[[1, 2, 3.1], [4, 5, 6], [7, 8, 9]],
                [[1, 2, 3], [4, 5, 6], [7, 8, 9]]]) #
3D array (a "tensor")
```

Key attributes every array has:

Attribute	Meaning
.shape	The size along each dimension, e.g. (2, 3) = 2 rows, 3 columns
.ndim	The number of dimensions (1D, 2D, 3D, ...)
.dtype	The data type of the elements (e.g. int64, float64)
.size	The total number of individual elements in the array

```
print(a3d.shape) # (2, 3, 3)
print(a1d.shape) # (3,)
print(a2d.shape) # (2, 3)
print(a3d.ndim) # 3 -> tells us it's a 3D array
print(a3d.dtype) # float64 -- because one value (3.1) was a
decimal
print(a2d.dtype) # int64 -- all values were whole numbers
print(a3d.size) # total individual elements
```

```
print(type(a3d))          # <class 'numpy.ndarray'>
```

NumPy arrays and Pandas are closely linked — a 2D NumPy array converts directly into a Pandas DataFrame:

```
import pandas as pd
df = pd.DataFrame(a2d)
```

3. Three Ways to Create Arrays

a) Arrays of ones or zeros (useful as starting templates / placeholders)

```
ones = np.ones((4, 4))    # a 4x4 array filled with 1.0
zeros = np.zeros((4, 3))  # a 4x3 array filled with 0.0

print(ones.dtype)        # float64 by default
print(type(ones))        # numpy.ndarray
```

b) A range of numbers with arange

```
range_array = np.arange(1, 21, 3)  # start=1, stop=21 (excluded),
step=3
# -> [1, 4, 7, 10, 13, 16, 19]
```

c) Random numbers

```
random_matrix = np.random.randint(0, 10, size=(3, 5))  # random
ints between 0-9, shape (3,5)
print(random_matrix.shape)  # (3, 5)
print(random_matrix.size)   # 15

rand_matrix = np.random.rand(3, 5)  # random floats between 0 and
1, shape (3,5)
print(rand_matrix.sum())           # sum of all elements
```

Reproducibility with seed: Random numbers are different every time you run the code. If you want to share your project and have others get the *exact same* “random” results, fix the seed:

```
np.random.seed(seed=0)
rand_matrix = np.random.rand(3, 5)  # will always generate the same
values when seed=0
```

4. Accessing Elements (Indexing)

Just like Python lists, you access elements using square brackets [], but NumPy extends this to multiple dimensions at once.

```
a3d[0]      # the first "layer" of a 3D array (a full 2D sub-array)
a2d[0]      # the first row of a 2D array
a2d[1]      # the second row
a3d[1]      # the second layer of the 3D array
```

5. Matrix Slicing

Slicing lets you pull out sub-sections of an array using start:stop for **each dimension**, separated by commas: array[dim0_slice, dim1_slice, dim2_slice, ...]

```
a3d[0:2, :1]          # rows 0-1 of the outer dimension, only the
```

```

first row of the next dimension
a3d[0:2, :1, :2]      # additionally: only the first 2 columns of
the innermost dimension
a3d[0:2, 0:1, 0:2]    # same as above, written explicitly
instead of using a shorthand ':'

```

This scales to any number of dimensions:

```

randomm = np.random.randint(10, size=(3, 4, 5))
randomm[0:3:2, 0:2, 0:3]    # note the "0:3:2" -> start:stop:step,
so it skips every other item

randomm4d = np.random.randint(10, size=(2, 3, 4, 5)) # a 4D array
randomm4d[:, :, 0:1, 0:1]    # take just the first slice of every
dimension

```

6. Array Manipulation — Arithmetic & Broadcasting

Broadcasting means NumPy automatically applies math operations **element-by-element**, matching up array shapes, without you writing a loop.

```

a1dd = np.array([1, 2, 3, 4])

a1dd + ones      # adds 1 to every element (element-wise / one-to-one
mapping)
a1dd - ones
a1dd * ones
a1dd / ones

```

This also works between two same-shaped arrays:

```

a3ddd = np.array([
    [[1, 2, 3, 3], [4, 5, 6, 4], [7, 8, 9, 5]],
    [[1, 2, 3, 7], [4, 5, 6, 8], [7, 8, 9, 1]]
])

t = a3ddd[:, 2, :3]
v = a3ddd[:, 2, :3]

t + v      # element-wise addition
t * v      # element-wise multiplication

```

Other arithmetic operators:

```

a1dd / a2ddd      # element-wise division
a1dd // a2ddd     # floor division -- rounds each result down to a
whole number
a1dd ** a2ddd     # element-wise exponentiation (power)
a1dd ** 2         # square every element
np.square(a1dd)   # same result, using a NumPy function instead
of **2
a2ddd % 2         # modulus -- remainder after dividing each
element by 2

```

7. Aggregations

An **aggregation** takes a large array of numbers and reduces it down to a single summary value (like a total).

```

sum(a1d)          # Python's built-in sum() -- works, but SLOWER on
large arrays
np.sum(a1d)       # NumPy's sum() -- optimized and much FASTER on
large arrays

```

The performance difference becomes obvious on large data:

```
lambI_array = np.random.random(1000)    # 1000 random floats
np.sum(lambI_array)

%timeit sum(lambI_array)                  # Python's sum() -- slower
%timeit np.sum(lambI_array)               # NumPy's sum() -- faster (uses
optimized C code internally)
```

Rule of thumb: always prefer `np.sum()`, `np.mean()`, etc. over Python's built-ins when working with NumPy arrays.

```
np.max(a2d)    # largest value in the array
np.min(a2d)    # smallest value in the array
```

8. Statistics

Function	Meaning
<code>np.mean()</code>	The average — sum of all values divided by how many values there are
<code>np.var()</code>	Variance — measures how spread out the values are from the mean
<code>np.std()</code>	Standard deviation — the square root of the variance (same units as the original data)
<code>np.mean(a2d)</code> <code>np.var(a2d)</code> <code>np.sqrt(np.var(a2d))</code> <code>np.std(a2d)</code>	<i># Mean</i> <i># Variance</i> <i># Std = square root of variance</i> <i># Same result, computed directly</i>

9. Matrix Multiplication

There are two very different kinds of “multiplication” for matrices:

a) Element-wise multiplication (*)

Multiplies matching positions together. **Requires matching shapes.**

```
matrix1 * matrix2    # each element multiplied by the element in the
same position
```

b) Dot product (.dot())

The proper mathematical matrix multiplication. **Shape rule: the number of columns in the first matrix must equal the number of rows in the second matrix** (dimensions don't need to match exactly like element-wise multiplication).

```
matrix1.dot(matrix3)    # matrix1 is (4,4), matrix3 is (4,3) ->
result is (4,3)
```

If the shapes don't line up for a dot product, use **transpose** (`.T`) to flip a matrix's rows and columns so the shapes fit:

```
matrix1.dot(matrix4.T)    # flips matrix4 so its shape becomes
compatible
```

10. Reshape and Transpose

Transpose (.T) swaps rows and columns:

```
matrix4 = np.random.randint(29, size=(4, 3))
matrix4.T           # shape flips from (4,3) to (3,4)
print(matrix4.shape) # (4, 3)
print(matrix4.T.shape) # (3, 4)
```

Reshape (.reshape()) changes an array's shape while keeping all the same data (the total number of elements must stay the same):

```
matrix4.reshape(2, 3, 2)           # reorganizes 12 elements into a
(2,3,2) 3D shape
matrix4.reshape(2, 3, 2).shape      # (2, 3, 2)
```

11. Practical Example — Combining NumPy + Pandas

This ties everything together in a real scenario: calculating weekly sales totals.

```
np.random.seed(seed=0)
matrix5 = np.random.randint(29, size=(5, 3)) # 5 days x 3 items --
random quantities sold

price = np.array([27, 53, 54]) # price per item (item1, item2,
item3)

# Turn the price array into a labeled 1-row DataFrame
price_df = pd.DataFrame(price.reshape(1, 3),
                        index=['Price'],
                        columns=['item1', 'item2', 'item3'])

print(price_df)
print(price_df.shape) # (1, 3)

# Dot product: (1x3) prices . (3x5) transposed quantities = (1x5)
total cost per day
total = price_df.dot(matrix5.T)
print(total)

# Turn the quantities into a labeled DataFrame, one row per day
weekly_sales = pd.DataFrame(matrix5,
                            index=['mon', 'tue', 'wed', 'thurs',
'fri'],
                            columns=['item1', 'item2', 'item3'])

print(weekly_sales)

# Add a new column with each day's total cost.
# total is shape (1,5) -- we transpose it to (5,1) so it lines up
with weekly_sales' 5 rows,
# and pull out the plain values with .values.flatten() so pandas can
assign it cleanly.
weekly_sales['Total ($)'] = total.T.values.flatten()
print(weekly_sales)
```

12. Comparison Operators

Just like arithmetic, comparisons are also broadcast element-by-element and return an array of True/False values.

```
a1d > a2d      # element-wise "greater than" -> array of True/False
a1d < a2d      # element-wise "less than"
a1d > 3        # compares every element against a single number
```

```
a1d <= a2d      # less-than-or-equal
a1d >= a2d      # greater-than-or-equal
a1d == a2d      # equality check, element-by-element
```

Regular math operators also broadcast the same way:

```
a1d / a2d
a1d // a2d
a1d * a2d
a1d % a2d
```

13. Sorting Arrays

```
matrix6 = np.random.randint(29, size=(5, 3))

np.sort(matrix6)                # sorts each ROW in ascending
order (default: last axis)
np.sort(matrix6, axis=0)        # sorts each COLUMN in ascending
order
np.sort(matrix6, axis=1)        # sorts each ROW in ascending
order (explicit)

np.argsort(matrix6)             # returns the INDEXES that
would sort the array, not the values
np.argmin(matrix6, axis=0)      # index of the smallest
value in each column
np.argmax(matrix6, axis=0)      # index of the largest
value in each column
```

14. Reading Images as Arrays

This is where NumPy connects directly to real-world data: **an image is just a NumPy array of numbers** (pixel values).

```
from matplotlib.image import imread
import matplotlib.pyplot as plt

img_data = imread('sample_image.png') # reads the image straight
into a NumPy array

plt.figure()
plt.imshow(img_data) # displays the array as an image
plt.show()

print(img_data.shape) # e.g. (height, width, channels) -- a "four
dimension"-like structure:
# height x width x RGB(A) color channels
```

Each pixel is represented by numbers (Red, Green, Blue, and sometimes Alpha/transparency), which is exactly why NumPy — a library for handling grids of numbers — is the natural way to work with images in Python.

Summary — Why This All Matters for Machine Learning

- **Arrays** are the universal format ML models expect data in.
- **Broadcasting** lets you transform entire datasets in one line instead of slow loops.
- **Aggregations & statistics** (mean, std, variance) are used constantly for understanding and normalizing data.
- **Matrix multiplication (dot product)** is the literal mathematical

operation behind how neural networks and linear models make predictions.

- **Reshaping** data (e.g., flattening an image, or reorganizing a dataset) is a routine preprocessing step before feeding data into a model.

What's Next

With Python fundamentals and NumPy covered, the natural next step is **Pandas** (DataFrames, cleaning real datasets), then **Matplotlib/Seaborn** (visualization), then your first **Scikit-learn** model.